IHG HOSPITALITY

DEPLOYMENT AND OPERATIONS

Deployment Guide

How the Enterprise Hospitality, Restaurant & Food Delivery Management Platform is hosted, configured, released and kept running.

VERSION
1.0

DATE
4 October 2026

PREPARED FOR
International Hospitality Group

PREPARED BY
Karimul Islam
Batch 26/5
AI-Powered Software Engineering, ICT Bangladesh

LIVE WEBSITE
ihg-hospitality.netlify.app

STATUS
Issued for acceptance review

DOCUMENT CONTROL

About this document

This guide explains how the Enterprise Hospitality, Restaurant & Food Delivery Management Platform is hosted, configured, released and kept running. It is written for the person who takes over the platform after hand-over: someone comfortable with a web dashboard and a terminal, but with no prior experience of these particular services.

Item	Detail
Title	Deployment & Operations Guide
Version	1.0
Date	4 October 2026
Status	Issued for acceptance review
Applies to	The release deployed from branch `master`; database migration `e5f1c92b4d73`
Companion document	User Manual (guest and staff instructions)

Version	Date	Change
1.0	4 October 2026	First release, covering the live deployment.

How to read this guide

You will see	It means
Note	Background that helps you understand a step.
Tip	A time-saver, or a way to avoid a delay.
Caution	A step where a small mistake is easy to make.
Important	Something that can cause harm or loss if ignored.
Dark code blocks	Commands to type exactly as shown. Text in capitals, such as `PASTE_THE_FULL_NEON_CONNECTION_STRING_HERE`, is a placeholder you replace.
Dashed screenshot frames	Marked spaces for screenshots, each saying which screen to capture. Hide any secret before saving an image.

Contents

01 **Introduction**
What is deployed, where it lives, and who should read what.

02 **Architecture**
The five services, how they connect, and why each was chosen.

03 **Before you begin**
Accounts, tools and the layout of the repository.

04 **Configuration reference**
Every setting the platform reads, and where it is set.

05 **Step-by-step deployment**
Database, email, backend and website, in the order to build them.

06 **Verification checklist**
Fourteen checks that prove the deployment works end to end.

07 **Operating the platform**
Releasing changes, migrations, backups, rollback and key rotation.

08 **Free-plan behaviour**
What to expect from free hosting, and how Alice degrades gracefully.

09 **Security and data protection**
Secrets, access control, transport security and a launch checklist.

10 **Troubleshooting**
Symptoms seen during this project, their causes and fixes.

11 **Limitations and next steps**
What is deliberately a demo, and what to change before real use.

A–C **Appendices**
Command reference, release checklist and glossary.

CHAPTER 01

Introduction

The platform joins three businesses that normally run separately: a hotel, a restaurant and a food-delivery service. It keeps their records in one place, so a booking, a table, an order and a delivery are all connected to the same guest account. This guide covers the **technical side**: how that system is put on the internet and looked after.

5
hosted services working together

$0
monthly cost on the free plans used

1
git push releases a change

HTTPS
on every connection

1.1 Live addresses

Purpose	Address
Public website (guests and staff)	https://ihg-hospitality.netlify.app
Backend API	https://ihg-management-platform.onrender.com
API documentation (interactive)	https://ihg-management-platform.onrender.com/docs
Health check	https://ihg-management-platform.onrender.com/health
Source code	https://github.com/Karim-1211/ihg-management-platform

The first visit can be slow
The free backend sleeps after about 15 minutes without traffic, and the first request after that takes roughly 30 to 60 seconds. Open the website a minute before a demonstration.

1.2 Where to start

Choose by role
Reviewing the deployment: Chapters 2, 6 and 11.
Taking over the code: Chapters 3, 4, 5 and 7.
Running it day to day: Chapters 7, 8 and 10, and Appendix B.
Responsible for data protection: Chapter 9.

CHAPTER 02

Architecture

The platform is made of five hosted services. The website is a static application served by Netlify. It talks to an API running on Render, which keeps its data in a PostgreSQL database on Neon, sends email through Brevo, and calls AI providers for Alice, the concierge. GitHub holds the code and triggers both builds.

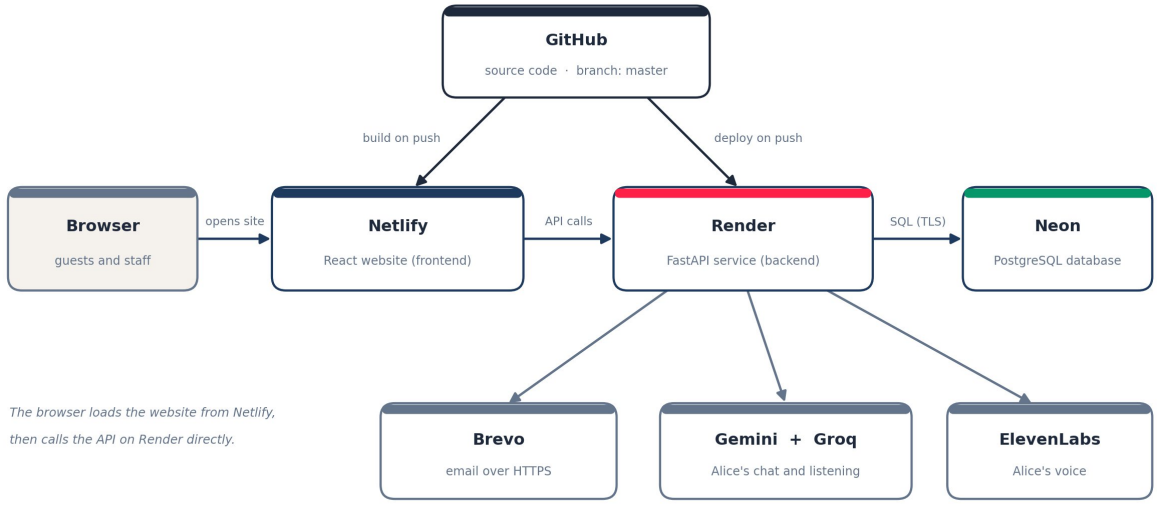


Figure 1 *How the hosted services connect*

2.1 Components

Component	Technology	Hosted on	Responsibility
Website	React, Vite, Tailwind CSS	Netlify	Everything guests and staff see. Calls the API from the browser.
API	FastAPI (Python 3.12), SQLAlchemy, Alembic	Render web service	Sign-in, roles, bookings, orders, billing, inventory, notifications, business rules and AI orchestration.
Database	PostgreSQL	Neon	All records: users, rooms, bookings, orders, invoices, stock, rules, audit trail.
Email	Brevo transactional API	Brevo	Confirmations, invoices (with PDF attached), welcome emails.
Alice: chat and listening	Gemini, Groq (Whisper)	Provider APIs	Answers questions, and turns speech into text.
Alice: voice	ElevenLabs, then the browser voice	Provider API / browser	Speaks Alice’s replies.
Source and release trigger	Git	GitHub	Single source of truth. A push to `master` starts both builds.

2.2 How one request travels

1. The guest opens the website. Netlify serves the pre-built pages.
2. The page asks the API for data, using the address stored in `VITE_API_URL` at build time.
3. The API checks the guest's login token, applies the business rules, and reads or writes the database on Neon.
4. If an email is needed, a background worker sends it through Brevo without delaying the guest.
5. If the guest talks to Alice, the API calls Gemini or Groq for the answer and ElevenLabs for the voice.

2.3 Why these services were chosen

Decision	Reason
Database on Neon rather than on Render	Render's free PostgreSQL database expires after 30 days. Neon's free database does not, so records survive.
Email through Brevo's web API	Render's free web services block the standard outbound mail ports (25, 465 and 587), so ordinary SMTP cannot be used. Brevo works over HTTPS, which is never blocked.
Website on Netlify	The site carries heavy static files (photographs, Alice's audio clips). Serving them from Netlify keeps that traffic off the API server.
Local voice engine excluded from production	The engine used on the developer machine needs far more memory than a free server has. Production uses ElevenLabs, with the browser's own voice as the final fallback.

> **Accurate at the time of writing**
> Free-plan terms change. The reasons above reflect each provider's published terms in October 2026; re-check them before relying on a free plan for a real business.

CHAPTER 03

Before you begin

3.1 Accounts

Account	Used for	Plan
GitHub	Stores the source code and triggers deployments	Free
Neon	The production database	Free
Render	Runs the backend API	Free
Netlify	Hosts the website	Free
Brevo	Sends email	Free
Google AI Studio	Gemini keys for Alice's chat	Free
Groq	Chat fallback and speech-to-text for Alice	Free
ElevenLabs	Alice's spoken voice	Free monthly credits

3.2 Tools on the developer computer

- **Git**, to commit and push changes.
- **Python 3.12**, to run the backend and the maintenance scripts.
- **Node.js 22**, to run and build the website locally.
- **PostgreSQL client tools** (`pg_dump` and `psql`), to copy and back up the database.
- A code editor, and a password manager for the secrets listed in Chapter 4.

3.3 Repository layout

Path	Contains
`backend/`	The FastAPI application (`app/`), database migrations (`alembic/`), `requirements.txt`, and the seed and maintenance scripts.
`frontend/`	The React application (`src/`) and public files: images, Alice's audio clips and the brand logo.
`netlify.toml`	Build settings for the website, and the rule that makes page refreshes work.
`.gitignore`	Keeps secrets, database backups and local files out of Git.

Secrets never go into Git

The `.env` file and any `*.sql` backup are listed in `.gitignore`. Before every commit, run `git status` and confirm neither appears. A leaked key or database password must be treated as compromised and replaced (see Chapter 7).

CHAPTER 04

Configuration reference

The platform has no settings in its source code. Everything is read from **environment variables**: named values set in each service's dashboard. This chapter lists them by name and purpose. Values marked secret must be created and stored by the administrator and never written into a document, a chat or a commit.

4.1 Backend (set on Render)

Variable	Required	Purpose
`PYTHON_VERSION`	Yes	Python version Render uses. Currently `3.12.10`.
`APP_ENV`	Yes	Set to `production` to switch on the startup safety checks.
`DEBUG`	Yes	Must be `false` in production.
`DATABASE_URL`	Yes, secret	The Neon connection string, copied exactly as Neon shows it.
`JWT_SECRET_KEY`	Yes, secret	Signs login tokens. A long random value, different from the one used locally.
`ACCESS_TOKEN_EXPIRE_MINUTES`	Optional	How long a login lasts. Default 30.
`CORS_ALLOWED_ORIGINS`	Yes	The website address allowed to call the API: the Netlify address with no trailing slash.
`EMAIL_MODE`	Yes	`SMTP` sends real email; `DEMO` only logs it.
`BREVO_API_KEY`	For email, secret	Lets the API send through Brevo.
`SMTP_FROM_EMAIL`	For email	The sender address, which must be verified in Brevo.
`SMTP_FROM_NAME`	Optional	Sender display name, e.g. IHG Grand Dhaka.
`GEMINI_API_KEY_1`, `GEMINI_API_KEY_2`	Yes, secret	Alice's chat. Two keys give a fallback if one runs out.
`GROQ_API_KEY`	Yes, secret	Chat fallback, and speech-to-text for Alice's microphone.
`ELEVENLABS_API_KEY`	Recommended, secret	Alice's spoken voice. Without it she uses the browser voice.
`ELEVENLABS_VOICE_ID`	Optional	Chooses which ElevenLabs voice speaks.
`TTS_PROVIDER_ORDER`	Optional	Order in which voice providers are tried. Default `kokoro,elevenlabs,gemini,groq`.
`GROQ_STT_MODEL`	Optional	Speech-to-text model. Default `whisper-large-`

Variable	Required	Purpose
		`v3-turbo`.
`AI_MODEL_NAME_GEMINI`, `AI_MODEL_NAME_GROQ`	Optional	Override the chat model names if a provider renames them.

4.2 Website (set in netlify.toml)

Variable	Purpose
`VITE_API_URL`	The API address with `/api/v1` on the end. Baked into the website when it is built.
`VITE_ALICE_USE_CLIPS`	Set to `false` so Alice speaks every line in one hosted voice instead of mixing recorded clips with live speech.
`NODE_VERSION`	Node.js version used for the build: `22`.

> **Website settings need a rebuild**
> Because these values are built into the site, changing one means pushing a commit or choosing “Clear cache and deploy site” in Netlify. Backend variables take effect when Render restarts, which happens automatically when you save them.

4.3 Local development only

The developer’s own `backend/.env` points at a local database and, optionally, sends mail through a Gmail account using `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD` and `SMTP_USE_TLS`. It must **not** contain `BREVO_API_KEY`, which is what keeps local email on Gmail. Use different secrets locally and in production so a laptop mishap cannot expose the live site.

4.4 The startup safety check

When `APP_ENV` is `production`, the API checks its own settings at startup and writes a **Production readiness** warning to the log for each problem: debug left on, a placeholder secret, CORS still pointing at the local address, email in demo mode, or email selected without a sender. After a clean deployment the log shows none of these.

CHAPTER 05

Step-by-step deployment

Build the services in this order: database first, then email, then the backend, then the website, then connect them. Each section ends with a check, so a mistake is caught where it happens.

5.1 What makes the code deployable

- `requirements.txt` lists **only what the application imports**. Heavy local-only libraries (the local voice engine and its machine-learning stack) are deliberately absent, so the install is small enough for a free server.
- The settings loader converts Neon's `postgresql://` address into the form the database driver needs, so the string can be pasted exactly as Neon provides it.
- Email goes through Brevo's web API whenever `BREVO_API_KEY` is present, and through Gmail SMTP otherwise.
- The local voice engine is imported only if it is installed, so the API starts normally without it.
- `netlify.toml` holds the website build settings and the page-refresh rule.

5.2 Create the database on Neon

1. Sign up at neon.tech with the GitHub account.
2. Create a project:
   - Name `ihg-platform`.
   - Region **AWS Europe (Frankfurt)**, close to the backend.
   - The newest PostgreSQL version offered.
3. Choose **Connect**. Turn **Connection pooling off**, then copy the connection string into the password manager.
4. Export the existing database from the developer computer (see below).
5. Import it into Neon (see below), then run the check.

```
# 1. export (the file is saved outside the project folder)
pg_dump -U postgres -h localhost -d ihg_management --no-owner --no-privileges -f
"$HOME\Desktop\ihg_backup.sql"

# 2. import into Neon
psql "PASTE_THE_FULL_NEON_CONNECTION_STRING_HERE" -f "$HOME\Desktop\ihg_backup.sql"

# 3. check the copy arrived
psql "PASTE_THE_FULL_NEON_CONNECTION_STRING_HERE" -c "SELECT (SELECT count(*) FROM users)
AS users, (SELECT count(*) FROM rooms) AS rooms, (SELECT version_num FROM
alembic_version) AS migration;"
```

SCREENSHOT

Neon: the Connect dialog with pooling switched off

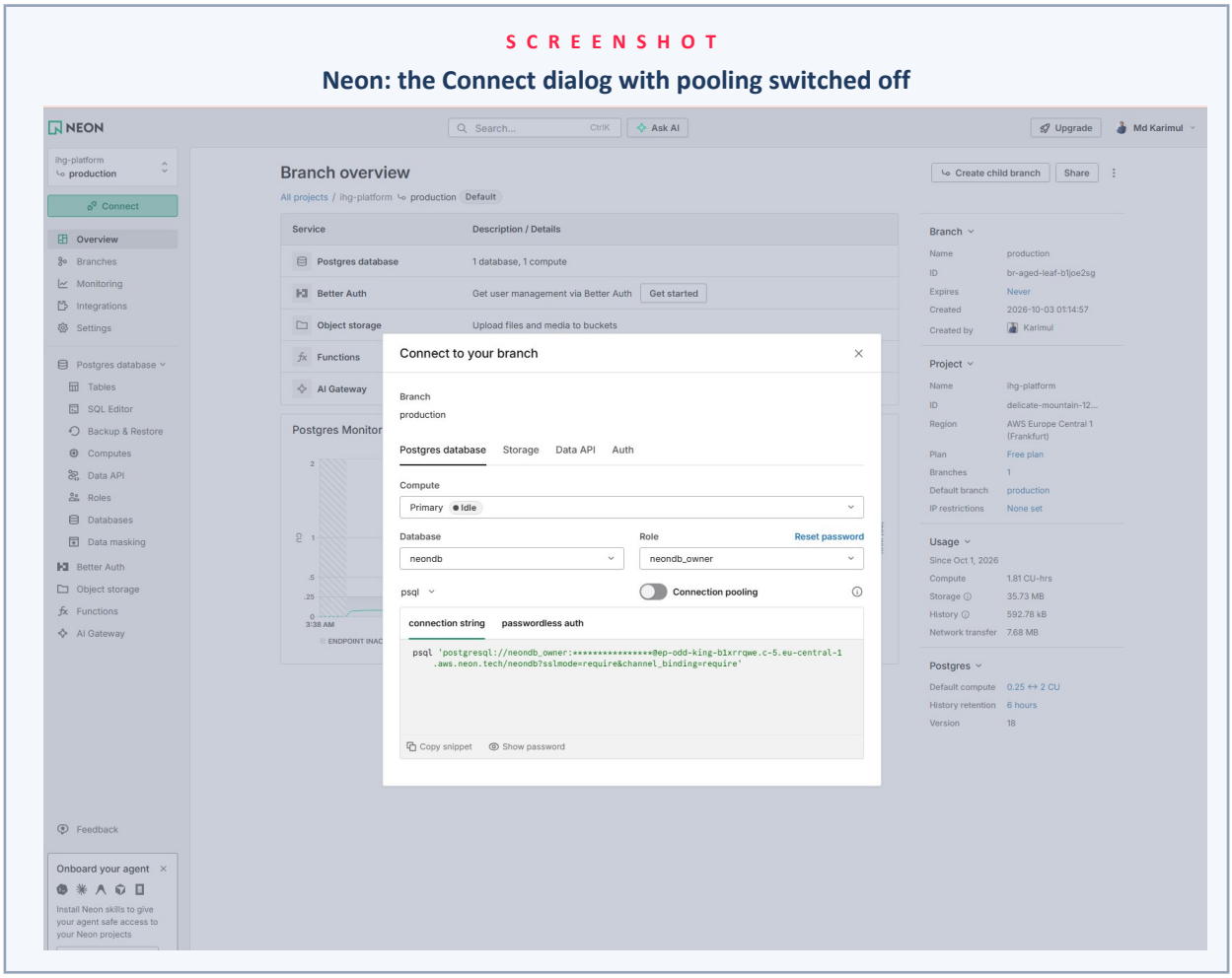


Figure 2 *Neon: the Connect dialog with pooling switched off*

Handle the backup like a password

The export contains password hashes and guest details. Keep it outside the project folder (the `*.sql` rule in `.gitignore` is a safety net, not a plan), and delete it when it is no longer needed.

5.3 Set up email on Brevo

1. Create a free account at brevo.com.
2. Add and **verify a sender**: the address that will appear as the “From” address. Brevo emails a confirmation link to it.
3. Create an **API key** under the SMTP & API section and store it in the password manager.
4. Under Security, make sure **authorised IP restrictions are off**. Render’s free servers have no fixed address, so an allow-list would block every send.

5.4 Create the backend on Render

1. In Render choose **New → Web Service**, then connect GitHub and pick the repository.
2. Fill in the settings below.
3. Add the environment variables from section 4.1.

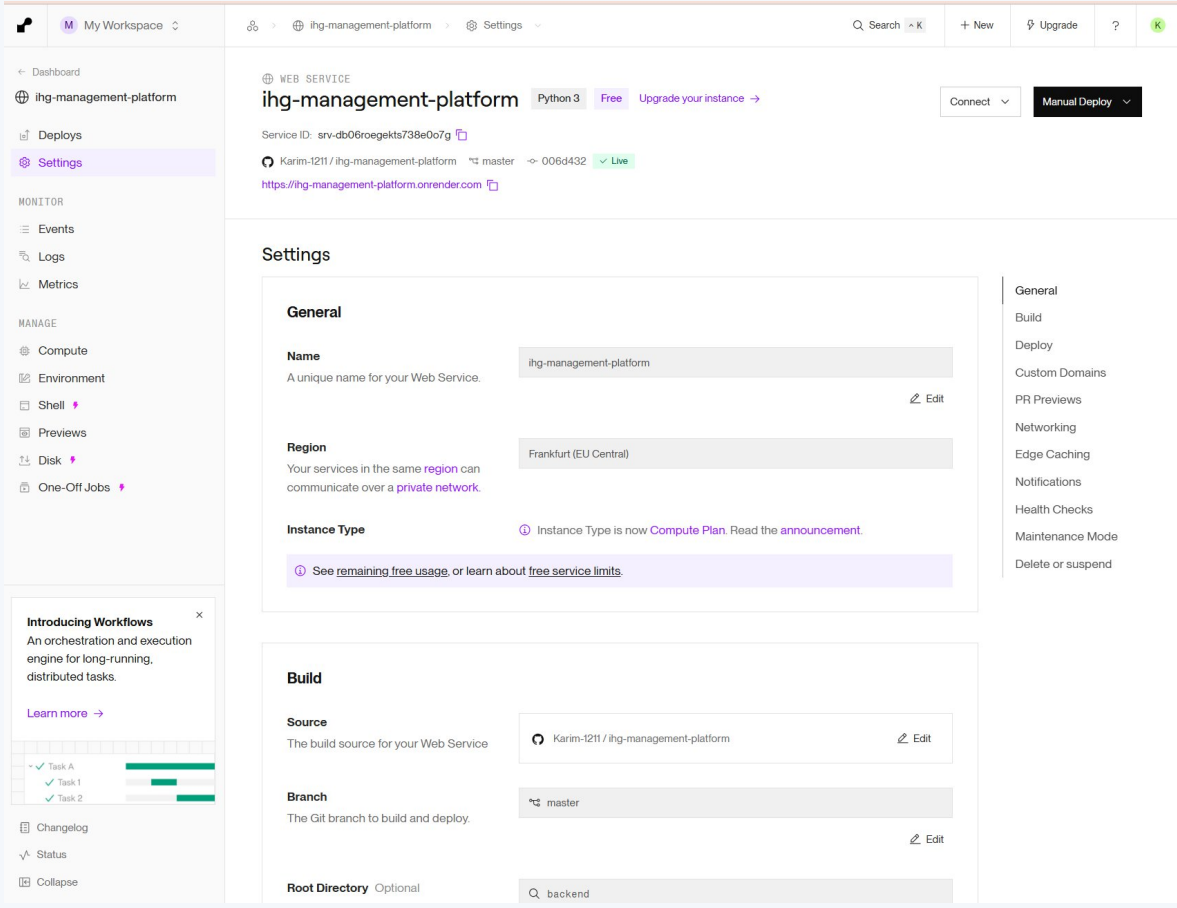
4 Under Advanced, set the **Health Check Path** to `/health`, then choose **Create Web Service**.

5 Wait for the build (three to five minutes) to end with “Your service is live”.

Setting	Value
Name	Any name; it becomes part of the address
Region	Frankfurt (EU Central)
Branch	`master`
Root Directory	`backend`
Language	Python 3
Build Command	`pip install -r requirements.txt`
Start Command	`uvicorn app.main:app --host 0.0.0.0 --port $PORT`
Instance Type	Free
Health Check Path	`/health`

SCREENSHOT

Render: service settings (Root Directory, commands, health check)



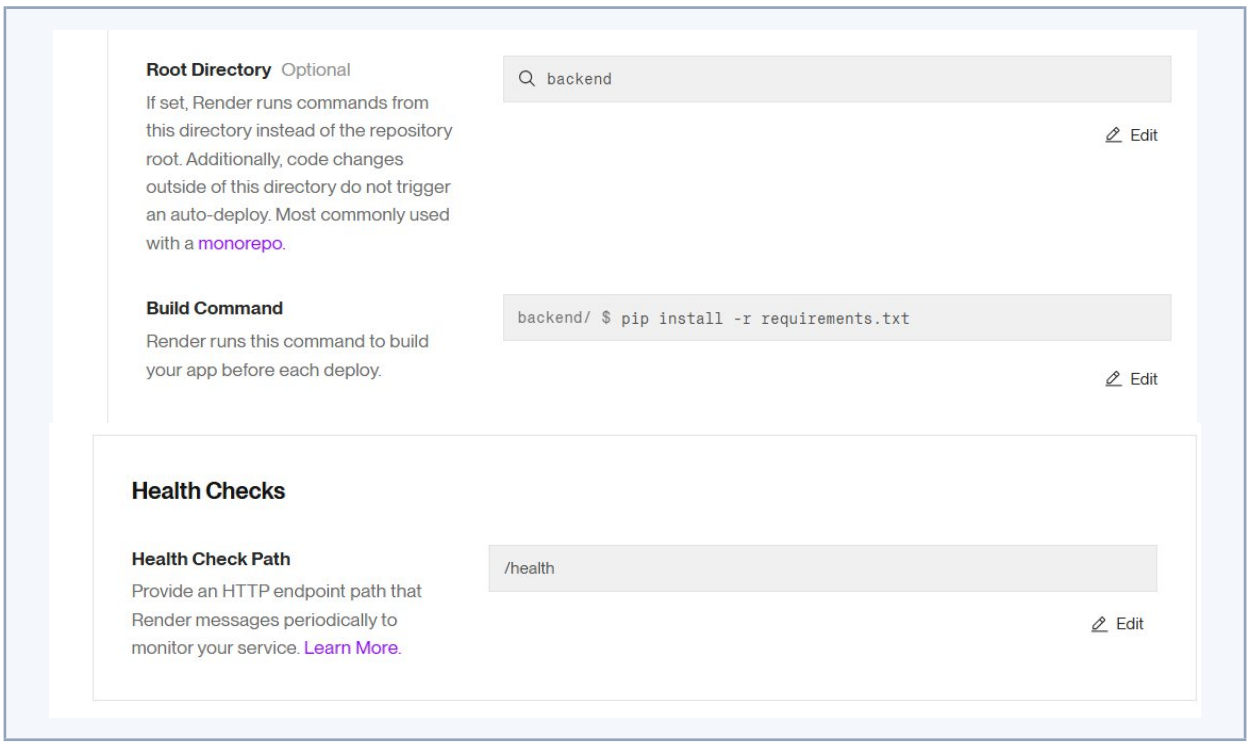


Figure 3 *Render: service settings (Root Directory, commands, health check)*

> **The most common first-build failure**
> If the build stops with “Could not open requirements file”, the **Root Directory** is empty. Set it to `backend`, save, and redeploy. Without it Render looks for `requirements.txt` in the wrong folder.

> **Why the start command does not migrate the database**
> The Neon database was copied from a database already at the latest migration, so nothing needs migrating at startup. Future migrations are applied deliberately, as described in section 7.2.

Check: open `/health` on the new address. It should show `"status":"healthy"` and `"database":"up"`.

5.5 Create the website on Netlify

The repository already contains `netlify.toml` at its root. It tells Netlify how to build the site and which API to talk to:

```
[build]
  base = "frontend"
  command = "npm run build"
  publish = "dist"

  [build.environment]
    NODE_VERSION = "22"
    VITE_API_URL = "https://<your-api-address>/api/v1"
    VITE_ALICE_USE_CLIPS = "false"

[[redirects]]
  from = "/*"
  to = "/index.html"
```

```
status = 200
```

1. In Netlify choose **Add new site → Import an existing project → GitHub** and pick the repository.
2. Confirm Netlify shows base `frontend`, build command `npm run build` and publish directory `frontend/dist`, then choose **Deploy**.
3. When it finishes, open **Site configuration → Change site name** and choose a readable name. Copy the final address.

> **What the redirect rule is for**
> The website is a single-page application. Without the rule, refreshing a page such as `/hotels/…` makes Netlify look for a file that does not exist and show its own 404 page.

5.6 Connect the website to the API

The API only accepts requests from addresses it has been told to trust. Until this step, the browser blocks every call from the live website.

1. In Render open the service → **Environment**.
2. Set `CORS_ALLOWED_ORIGINS` to the Netlify address, exactly, with **no slash at the end**.
3. Save. Render redeploys in about two minutes. Wait for “Live”.

CHAPTER 06

Verification checklist

Work through these in order on the live website. The first load may take up to a minute while the backend wakes. Stop at the first failure and use Chapter 10.

#	Check	Expected result	Pass
1	Open `/health` on the API address	`"status":"healthy"` and `"database":"up"`	☐
2	Open `/docs` on the API address	The interactive API documentation loads	☐
3	Open the website at `/hotels`	Hotel page with photographs; nine rooms, numbered 101 to 402	☐
4	Open a room, then refresh the browser	The page reloads normally (no Netlify 404)	☐
5	Sign in with a guest account	Signed in; My Account opens	☐
6	Ask Alice a question	She replies in text and in a spoken voice	☐
7	Book a room for future dates	Confirmation and invoice; booking appears in My Account	☐
8	Check the booking email	Branded confirmation arrives (check spam on the first send)	☐
9	Download the invoice as PDF	PDF opens with correct totals	☐
10	Sign in as staff and open Bookings	The new booking is listed	☐
11	Open Staff → Email Log	The confirmation shows as sent	☐
12	Place a delivery order and open the tracker	Tracker shows the order; the kitchen sees a ticket	☐
13	Subscribe through the popup or footer	A welcome code is shown and an email arrives	☐
14	Read the Render log after startup	No “Production readiness” warnings	☐

Before any demonstration

Run checks 1, 3 and 6 a few minutes beforehand. They wake the backend and confirm the database, the website and Alice are all responding.

CHAPTER 07

Operating the platform

7.1 Releasing a change



Figure 4 *From a code change to the live site*

1. Make and test the change locally.
2. Run `git status`. Confirm that no `.env` file and no `.sql` file is listed.
3. Commit and push to `master`.
4. Wait for Render to show **Live** and Netlify to show **Published**.
5. Check `/health`, then reload the website with **Ctrl + Shift + R** so the browser drops its cached copy.

```
git status
git add -A
git commit -m "Describe the change"
git push
```

7.2 Changing the database structure (migrations)

Database structure is managed with Alembic. Render does not apply migrations when it starts, so they are applied deliberately, against Neon, from a developer computer:

1. **Back up first** (section 7.4).
2. In PowerShell, in the `backend` folder, point the session at Neon for this window only.
3. Run the migration, then clear the setting.

```
$env:DATABASE_URL = 'PASTE_THE_FULL_NEON_CONNECTION_STRING_HERE'
alembic upgrade head
Remove-Item Env:DATABASE_URL
```

> **Always clear the variable afterwards**
> While `DATABASE_URL` is set in a window, every Python command in that window uses Neon. Clearing it (or closing the window) returns you to the local database.

7.3 Seed and maintenance scripts

These scripts live in `backend/`. Each is safe to run more than once. To run one against the live database, set `DATABASE_URL` as in section 7.2 first.

Script	What it does
`seed_inventory_and_recipes.py`	Creates the ingredients and the recipe for each menu dish, so orders deduct stock.
`seed_tables_and_drivers.py`	Adds restaurant tables and delivery riders, and links riders to the restaurant.
`seed_room_details.py`	Fills room descriptions, views, beds and amenities, and the hotel's facilities list.
`seed_room_character.py`	Writes the per-room headline file used by the room detail panel.
`seed_promo_rule.py`	Creates the subscriber-discount business rule (default 10%).
`seed_delivery_rules.py`	Creates the kitchen-preparation and rider-travel minutes behind delivery estimates.
`fix_roomtype_images.py`, `fix_roomtype_primary.py`	Make sure every room type has an active, primary photograph.
`update_user_email.py`	Lists accounts and changes a user's login and notification email.
`cleanup_test_orders.py`	Removes test orders. Shows what it would do unless run with `--delete`.

7.4 Backups and restoring

```
# back up the live database
pg_dump "PASTE_THE_FULL_NEON_CONNECTION_STRING_HERE" --no-owner --no-privileges -f
ihg_live_backup.sql

# restore into an EMPTY database
psql "PASTE_CONNECTION_STRING_OF_THE_EMPTY_DATABASE_HERE" -f ihg_live_backup.sql
```

- Back up before every migration and before every demonstration.
- Store backups outside the repository, and treat them as personal data (Chapter 9).
- Test a restore into a spare database at least once, so the procedure is known to work.

7.5 Rolling back

What went wrong	Rollback
A code change broke the site	Undo it with `git revert <commit>` and push. Or redeploy an earlier successful deploy from the Render Events page and the Netlify Deploys page.
A migration or script damaged data	Restore from the backup taken beforehand (section 7.4).

What went wrong	Rollback
A setting was changed by mistake	Set the variable back in the dashboard and save; the service restarts by itself.

7.6 Changing a secret

1. Create the new value (a new API key, or a new database password in Neon).
2. Put it in the matching variable on Render and save. The service restarts.
3. Confirm `/health` is healthy, then revoke the old value at its source.

- Changing `JWT_SECRET_KEY` signs every user out. This is expected.
- Changing the Neon password means updating `DATABASE_URL` immediately afterwards, or the API loses its database.

7.7 Where to look when watching the system

Place	Shows
Render → Logs	Startup warnings, errors, and Alice's speech-to-text decisions (`STT segment kept/dropped`).
`/health`	Whether the API and the database are up.
Staff → Email Log	Every email, its status and the reason for any failure.
Staff → Audit Log	Who changed what: rules, service requests and other sensitive actions.

CHAPTER 08

Free-plan behaviour

Every service is on a free plan, which is right for a demonstration and for early use. These are the behaviours to expect, and none of them is a fault.

Service	Behaviour	What it means in practice
Render (API)	Sleeps after about 15 minutes idle; wakes on the next request in roughly 30 to 60 seconds	Open the site a minute early for a demo. The first click after a quiet spell is slow.
Render (API)	Blocks outbound mail ports	Email must use Brevo's web API, which is already configured.
Neon (database)	May pause when idle and resume on the next connection	The first query after a long quiet spell can be slower. The API reconnects automatically.
Brevo (email)	A daily sending allowance on the free plan	Plenty for a demo; check the current figure on Brevo before heavy use.
Groq (Whisper)	A free daily allowance of transcriptions, of the order of two thousand a day at the time of writing	Far more than a demonstration needs.
ElevenLabs (voice)	Monthly credits	When they run out, Alice switches to the browser's voice for the rest of that visit and returns to ElevenLabs automatically once credits are available again.
Gemini (chat)	Free quota per key	Two keys, then Groq, are tried in turn before Alice says she is busy.
Netlify (website)	Monthly build and bandwidth allowances	Ample for this scale.

8.1 How Alice degrades gracefully

Alice never depends on a single provider. Her voice uses the first one that answers, and her listening is handled by the platform itself rather than the visitor's browser:

- **Voice:** ElevenLabs, then (if configured) Gemini or Groq, then the browser's built-in voice. Once a provider fails during a visit, the whole visit stays on the next one, so a conversation never alternates between two different voices.
- **Listening:** the visitor's voice is recorded in the page and transcribed on the API by Whisper, so it works in any modern browser. Text produced from silence is filtered out using Whisper's own confidence scores.
- **Chat:** Gemini key 1, Gemini key 2, then Groq.
- **Everything else:** if all voice or listening services fail, Alice invites the visitor to type, and the rest of the platform is unaffected.

CHAPTER 09

Security and data protection

The proposal requires that guest and customer data stay confidential and that each person sees only what their job needs. This chapter records how the platform meets that, and what the administrator must keep doing.

9.1 What the platform already does

Area	Measure
Passwords	Stored only as Argon2 hashes, never in readable form.
Sign-in	Signed login tokens that expire (30 minutes by default) and are signed with a secret known only to the server.
Permissions	Every staff screen and API route checks the user's role. A kitchen user cannot open the bookings board; a receptionist cannot change business rules.
Transport	The website and API are served over HTTPS; the database connection requires TLS.
Cross-site access	The API accepts calls only from the configured website address.
Audit trail	Sensitive actions (such as changing a business rule) are written to the Audit Log with who and when.
Payments	Demonstration mode only: no real card or PayPal details are collected or stored.
Repository	`.env` files and database backups are excluded from Git.

9.2 What the administrator must do

- Keep every secret in a password manager. Never paste a key, token or connection string into a chat, an email or a ticket. When asking for help, remove the secret first.
- Use a different `JWT_SECRET_KEY` and different database credentials in production than on the developer computer.
- Rotate any secret that has been exposed, following section 7.6.
- Treat database backups as personal data: encrypt or store them securely, and delete old ones.
- Give staff accounts only the role they need, and deactivate accounts when people leave.

9.3 Points to know before real use

> **Alice's voice and listening are open to visitors**
> So that people can use Alice before they have an account, the speech and transcription endpoints do not require a login. This is deliberate, but it means anyone who finds the API address can use the free voice and transcription allowance. Add rate limiting before any public launch.

Demo accounts and demo policies

The database contains demonstration accounts with simple passwords, and business policies (tax rate, check-in times, cancellation window, discount percentage) that are assumed values. Change every demo password, and replace the policies with the real ones in Staff → Business Rules, before the system holds real data.

9.4 Launch checklist

Item	Done
All demo account passwords changed or the accounts removed	☐
Production secrets differ from local secrets	☐
`git status` clean of `.env` and `.sql` files, and no secret in the Git history	☐
Real business policies entered in Staff → Business Rules	☐
Rate limiting added to the public voice and transcription endpoints	☐
Backups scheduled and one restore tested	☐
Real payment provider integrated in place of demonstration payments	☐
Email sender is a verified company domain rather than a personal address	☐

CHAPTER 10

Troubleshooting

Each row below was met during this project's own deployment. Start with the symptom, check the likely cause, apply the fix.

Symptom	Likely cause	Fix
Render build stops: "Could not open requirements file"	Root Directory not set	Settings → Root Directory = `backend`, then redeploy.
The API will not start: "cannot import name …"	A file was pushed into the wrong folder, or not pushed at all	Check the file sits in the same folder as its neighbours, commit and push again.
`/health` shows `"database":"down"`	Wrong or outdated `DATABASE_URL`; Neon paused or password reset	Re-copy the string from Neon's Connect dialog and save it on Render.
The site loads but shows "Network Error"; the browser console mentions CORS	`CORS_ALLOWED_ORIGINS` missing, mistyped or ends with a slash	Set it to the exact website address, no trailing slash, and save.
Refreshing a page gives a Netlify 404	The redirect rule is missing	Confirm `netlify.toml` has the `[[redirects]]` block and redeploy.
Emails do not arrive	Sender not verified; `SMTP_FROM_EMAIL` does not match it; IP restriction on in Brevo; message in spam	Open Staff → Email Log for the exact reason, then correct the matching setting.
Alice has no spoken voice	ElevenLabs credits used up or key missing	She falls back to the browser voice automatically. Check `ELEVENLABS_API_KEY` and the credit balance.
Alice's microphone never hears anything	Microphone permission blocked; `GROQ_API_KEY` missing	Allow the microphone for the site; confirm the key; look for `STT segment` lines in the Render log.
Alice answers things nobody said	An older backend version is still running	Confirm the latest Render deploy is Live and shows the newest commit; check the `STT segment` log lines.
A change is not visible after a push	Deploy still running, or the browser cached the old site	Wait for Live and Published, then press Ctrl + Shift + R.
A promo code says "That code isn't recognised"	The code was issued by a different database (local versus live)	Codes belong to one database. Subscribe again on the live site to get a live code.
A room-type card shows "No image available" on the live site only	The photograph fix was applied to the local database, not Neon	Run the two image scripts from section 7.3 against Neon.

CHAPTER 11

Limitations and next steps

The platform meets the proposal's functional scope, and the deployment is complete and repeatable. These items are deliberately left as demonstration choices, and are the natural next steps toward real use.

Area	Current state	Recommended next step
Hosting	Free plans, with a sleeping backend	A paid always-on backend plan removes the 30 to 60 second wake-up.
Payments	Demonstration mode only	Integrate a real payment provider behind the existing payment layer.
Business policies	Assumed values, editable in Business Rules	Replace with the real tax, cancellation, stay-length and discount policies.
Public voice endpoints	Open to all visitors	Add per-address rate limiting.
Email sender	A single verified address	Verify a company domain for better delivery and branding.
Custom address	Provider-generated web addresses	Attach a company domain to Netlify and Render.
Monitoring	Logs and the health endpoint	Add an uptime monitor and error alerts.
Data migration	Records start from the copied database	Use the planned import path (validate, preview, import) for historical paper records.

APPENDICES

Appendices

Appendix A. Command reference

Task	Command
See what has changed	`git status`
Release a change	`git add -A`, then `git commit -m "message"`, then `git push`
Undo a released change	`git revert <commit>`, then `git push`
Apply database migrations	`alembic upgrade head` (with `DATABASE_URL` set, section 7.2)
Start the API locally	`uvicorn app.main:app --reload` (from `backend`)
Start the website locally	`npm run dev` (from `frontend`)
Back up a database	`pg_dump "<connection string>" --no-owner --no-privileges -f file.sql`
Point a PowerShell window at Neon	`$env:DATABASE_URL = '<connection string>'`
Clear that setting	`Remove-Item Env:DATABASE_URL`

Appendix B. Release checklist

Before pushing	Done
Change tested locally in the browser	☐
`git status` shows no `.env` and no `.sql` file	☐
Database migration written and tested, if the structure changed	☐
Backup of the live database taken, if a migration is included	☐

After pushing	Done
Render shows Live and Netlify shows Published	☐
`/health` is healthy	☐
Verification checks 3, 5, 6 and 7 pass (Chapter 6)	☐
No new “Production readiness” warning in the Render log	☐

Appendix C. Glossary

Term	Meaning
API	The server program the website talks to. It holds the business rules and the only access to the database.
Cold start	The delay when a sleeping free server wakes for the first visitor.
CORS	A browser safety rule. The API lists which website addresses may call it.
Environment variable	A named setting held in a service's dashboard rather than in the code.
Health check	A small address (`/health`) that reports whether the API and database are working.
JWT	The signed token a user receives when they sign in; it proves who they are on each request.
Migration	A scripted change to the database's structure, applied in order and recorded.
Seed script	A script that fills the database with starting data.
Whisper	The speech-recognition model Alice uses to turn a recorded voice into text.